Q23: Agent Harnesses and Failure Attribution

Paradise (nuc13)

October 2026

The question

- "What is a harness, why does it matter, and how much of an agent failure comes from model vs. instructions vs. harness?"
- You are seeing one right now: Claude Code, Codex, Hermes = model + harness
- Answer preview: **the harness is the only layer you can change — and it routinely induces failures that look like model failures**

What a harness is

Everything around the model in an agentic loop:

1. **Context construction** — system prompt, files, history assembly
2. **Tool schemas** — what tools exist and how they are described
3. **Dispatch** — validating and routing calls to real code
4. **Error surfaces** — how failures are phrased back to the model
5. **Permission gates** — which actions need approval
6. **Stopping** — max turns, budgets, timeouts, completion detection

ReAct loop [**?**] is the skeleton; production harnesses are this loop plus engineering.

Why the harness matters

- It is the *only* layer most teams can change: model is fixed, tools are fixed, harness is yours
- SWE-agent: an interface designed for the model (ACI) beat the human Linux shell on SWE-bench by a wide margin — **same weights** [**?**]
- Anthropic's agent guidance: successful deployments = simple, composable patterns + carefully designed tools, not exotic orchestration [**?**]
- Harness-layer changes can move performance by margins that look like model-generation jumps

Three suspects when an agent fails

- **Model failure**: wrong reasoning, hallucinated tool args — *given correct context and correct tool results*
- **Instruction failure**: the model did what it was told; the spec was wrong (ambiguous prompt, missing constraint, missing tool)
- **Harness failure**: truncated context, wrong tool result injected, timeout kills a correct run, gate blocks without explanation

Easy to state, hard to apply — because the layers *interact*.

Why attribution is hard: harness → model causation

- Opaque error surface ("error") ⇒ model retries blindly ⇒ *looks like* a dumb model, *is* an error-surface design
- Ambiguous tool schema ⇒ model passes plausible-but-wrong args ⇒ "hallucination" that the schema caused
- Reflexion works when feedback is informative [**?**] — informativeness is a harness property
- Transcript shows what the model did, **not what it was given**

Four concrete failure modes

- **Context rot**: turn-2 constraint buried under 40k tokens by turn 30; every injection individually correct
- **Tool-result injection errors**: model reasons flawlessly from false premises; found only by diffing raw vs. injected result
- **Retry storms**: uninformative errors + premature timeouts; reads as "model is dumb"
- **Silent schema drift**: renamed backend field, stale schema, task fails much later

Each sits on the model/harness boundary — which is exactly why they get misattributed.

Instrumenting attribution

1. **Log tool I/O**: exact context sent, raw tool result, *transformed* result injected — the diff exposes injection errors
2. **Replay**: re-run on logged contexts; splice in the correct tool result — if the run then succeeds, you proved a harness cause
3. **Ablation**: swap one layer at a time — model / instructions / harness version

Demo: `code/attribution_harness.py` — 3 configs, failure-attribution table, pass/fail assertions.

The honest summary

- The decomposition is a *model* of causation; real failures are often joint
- Attribution answers "which fix is cheapest," not "which layer sinned"
- Discipline: log everything $\rightarrow$ replay to confirm $\rightarrow$ ablate to attribute
- Teams default to blaming the layer they do not own; a swap table forces the evidence to name the layer

Reading

- Yao et al. 2023 (ReAct — the loop)
- Yang et al. 2024 (SWE-agent — the agent-computer interface result)
- Schluntz & Zhang 2024 (Anthropic, Building Effective Agents)
- Shinn et al. 2023 (Reflexion — feedback quality and self-correction); Kim et al. 2023 (RCI)